Video Transcripts

Module 20: Introduction to Machine Learning and Advanced Probability

Video 1: Introduction (05:53)

So, this week we're going to do an overview of machine learning methods. We don't have time to go into great depth, but we're going to cover some of the techniques that are used to learn. So, there are two kinds of machine learning techniques, supervised learning, and unsupervised learning. We're going to look at supervised learning, where we have a training set of data where we know the answers. And we're going to build a model, and now our model is going to be linear regression. So, we're going to put a straight line, if you like, through a number of points.

Now there's going to be some error between our model and this training data, and our goal is to adjust the parameters of the model so that we minimize the error. Now, there are various metrics of error, we're going to use least squares. Some of the others involve cross entropy and other metrics that can be used. But least squares is an interesting one in that it allows us to use an iterative method. And in machine learning, we almost always iterating so that we're trying to minimize this error and get to a good solution, build a good model.

Now, we saw in linear regression that we need to choose features that are correlated with the parameter that we're trying to predict. So, feature selection is essential part and you've already covered in the data science part that we've done in this course, how to choose features based on correlation coefficients. Now we're going to look at how we minimize this error. We're choosing least squares, and least squares has an interesting property in that the error is shaped like a bowl. It's a parabola. So, the further we are away from the solution, the higher up the sides of this parabola we are.

Now initially, we're going to have quite a large error and so we're way up the sides of this parabola and we need to get to the minimum where the error is a minimum. So, we need to come down this bowl and what we do is we find the gradient that point that we're at and we follow that gradient. Now, the problem is as we move down this bowl, the gradient changes. So, we don't want to move too far before we recalculate the gradient. So, some of the tricks, if you

© MIT xPRO® 2024. All rights reserved.

like of machine learning are to choose how far we go in a step, how far down do we go? Because if we choose a gradient, it's a straight line, but the parabola is curved. So, we're going to move off our parabola, if you like.

Now once we get close to the solution as well, there's a chance that we're going to overshoot and our error is going to increase. So, we need to monitor the error. And then as we get close to the solution, we're probably going to change the size of our steps and make them smaller and smaller so that we don't overshoot. So, gradient descent, and then we're going to look at Bayes, Naive Bayes sneak, which is called Naive Bayes. Now Bayes's theorem basically relates how two sets intersect. So, we've got set A and set B and they intersect. Now that intersection can be defined two ways. It can be defined as given that we're in A, then the probability that we're also in B. Or we can define it as, given that we're in B, the probability that we're also in A. Now, those two statements need to be equal, and that's what Bayes's theorem does. It takes advantage of exactly that, that those two statements are equal. Now, the way we're going to apply it is going to be to calculating how many rich people are actually happy.

And we're going to be going to be given the information about how many people in the population are happy and what proportion of those happy people are rich. And it's a small proportion, but what we're after is how many rich people are happy. So, you're going to apply Bayes theorem to calculate that, and you'll often hear it as given the prior probability, we calculate a posterior probability based on some other data that we're given. And we'll then going to apply Naive Bayes to spam messages, understanding which messages are spam and which are not. And then also to another problem where we're going to use an assumption that our data are about our features if you like, our data is Gaussian and we're going to use probability density functions as measures of the probabilities that we're going to plug into Naive Bayes. So, let's take a look now at starting on machine learning.

Video 2: The Dot Product (04:31)

We're going to be manipulating coordinates and we're going to represent things as vectors and sometimes as well as matrices. And so we need to be clear about the direction of the vectors because it's going to become important when we do dot products. Let's take the example of this line. So, Y equals, say, some constant, W0 plus W1 some slope times X. So, that would give us the equation of a line. Let's take a look how we write this in mathematics. So, in mathematics, I

© MIT xPRO® 2024. All rights reserved.

always think of vectors as being columns like this. So, when I write Y equals X.w, what I'm thinking of is that I'm thinking of it laid out like this, that X has two columns, one of them will be all ones, and the other column will be x1, x2, x3, and then the ws will also be a column vector. So, I can get the coordinates of Y if you give me the X coordinates like this and w, the two constants. And, I'll just do a dot product. So, my model, if you like, will my predictive model will be a dot product of X and W. But in NumPy, we think of the vectors as being horizontal.

So, we'll have our row of ones and we'll have our row of X coordinates. Now to get the Ys, we need to have W in front. But W as well will be a normal NumPy vector. It'll be a row vector. So, when we do the dot product, we get w0 times 1 plus w1 times x0, and that'll give us y0, and then y1 will be w0 again plus w1 times x1. So, in NumPy we'd write it this way, w.X equals Y or Y equals w.X. It doesn't matter as far as the X which way the equals sign goes, but it does matter notice that here we add X times w and here we have w times X as a dot product.

Now, check your understanding with this little piece of code here. So, here's my w. I'm choosing w0 as one and w1 as two. We're going to have five points. I create a vector, a row vector of ones, and I create an array of zero through four and I stack them. And I put the row of ones on top of the row of Xs.

So, I'm making it look like this and now to get my Ys, I do a dot product of W and X, and we'll print it out. And so here's my X, you'll see it's a two dimensional array. It has a row of ones and the second row is 0, 1, 2, 3, 4. And when we take the dot product of w.X, we'll get Ys as being this 1, 3, 5, 7, 9. So, make sure that you're clear about your vectors. We'll be using NumPy, so I think you should think of them this way, that they'll be laid out horizontally.

And then when we want to do a predictive model for Y, say, we'll be doing w.X. So, take a look at this piece of code and work through it, change the values. Try it the other way around. Try doing X.w and see what you get. It'll be wrong so be careful. Get this clear in your head and you won't have any trouble. OK, bye for now.

Video 3: Gradient Descent and the Line of Best Fit (11:16)

We're going to look at linear regression. Now this is built into the libraries, but here we're going to do it by hand first to understand how we can solve this problem. So, typically we have an independent variable X that we're making measurements at, and these are all independent

© MIT xPRO® 2024. All rights reserved.

measurements, that one measurement doesn't depend on the other. So, that's our assumption. And we have our dependent variable Y. And what we need to do is we're going to build a model that is in some sense a best fit. Given an independent variable, we can then predict the dependent variable.

So, for example, at this position out here, given this X, we would then be able to create a Y. Let's see how we do this in two dimensions. First so, just with X and Y, later on, we have many independent variables. We can do linear regression with many features that affect the dependent variable. So, we see that we can develop a model which we call ordinary linear regression where we have the equation Y equals w.X plus some error. And why we introduce the error is because when we have more than two points as we do here, it may be impossible to have zero error that this line will be a best fit line where we minimize the error according to some metric. So, we can write this equation out in this form in two dimensions. The X's are both the X coordinates, but also to represent the intercept we put a row of ones and so that's our matrix. And our predictive model will be w.X and that will give us our Y's. So, we can generate any Y given w's and the X's, we can generate the Y's. So, we have a predictive model. This can be solved analytically for smaller problems, but in general, in machine learning, we'll be using gradient descent. Now to do that, we must decide what we mean by error.

So, here we've got some plus errors, and here we have some minus errors they could cancel out. So, typically what we do is we'll square these differences. So, we'll take the difference between the actual Y coordinate and our predictive model. So, w.X, we'll square that and that will be our error term. Now, what does that error term look like? Well, because it's a squared, it looks like this in w0, so w0 along the bottom and w1 up. As w varies, we'll have an error that looks like this. So, we'll have some starting point here. We're starting at w0 is 0 and w1 is 0 . And then, we're trying to find out what w's give us the minimum error. So, we take the gradient so that'll be at right angles to these contour lines and we step down. And now here, there are other contour lines I'm not plotting, but it eventually finds a solution. Now, we hope it's the best solution, but we'll need an algorithm to find it. So, here's our algorithm that to calculate the gradient of the error with respect to w. So, both w's, w0 and w1, we can take the derivative. So, it was a squared.

So, with the two will come down in front, then we'll have Y minus w.X, and then we need to because it's with dW, we'll get another X outside. So, this will be our equation for the gradient.

© MIT xPRO® 2024. All rights reserved.

And now what we do is we'll change the w's, according to that gradient, we'll go in the opposite direction. So, if the gradient happens to be out, we want to go down and make it less, so we make it a minus. So, we'll take a delta gradient. And what we'll do to get that is we'll multiply the gradient by what we call a learning rate. It'll be a number less than one usually. So, our new w's will be whatever the old w is, minus some learning rate times the gradient. Okay, let's take a look at the code. We saw how to set up the X matrix. So, here we're just going to generate some points as a test case. And what I'm going to do is then I generate. So, these are the X's. Then I generate a number of ones, a row of ones, and stack them up. We have the ones first, then the X's. Now, I'm choosing my w to be 0, 0. So, here we are and this was what my Y would be. Everything would be 0. If w0 is 0 and w1 is 0, there's no slope and the intercept is 0. So, that's going to be my starting point. But what I'm going to do is I'm going to add 1 to the Y's. So, now this is my solution that I'm looking for. I'm going to be starting down here with this guess, but then iterating towards this solution. So, here's the code in action. So, it starts here, the first iteration it gets to this point, so it increases the w0, so that's the intercept, and it increases the slope.

You see here that it begins to overshoot with the slope and so it corrects that and eventually it gets the correct solution. Okay, so here's the actual code. Here I'm getting the gradient descent, so we'll go for a max number of iterations. We're calculating the change in w, so here's the learning rate, and here's the gradient function. So, we pass in w, and xy is a combination of X and Y. So, we're just putting it into a tuple, putting the X and the Y, and we'll just pass it around like that and then pull it out. So, here we see, when we're calculating the gradient, we take the xy and we separate the two. So, it's just a convenience. Then we get a predicted Y and we get the difference y - ypredicted. And remember, we need to multiply that by X. So, to get the gradient, the x is a vector, the difference is a scalar. So, the gradient will be, it'll give us a one-dimensional vector with two entries, the gradient of w0 and w1. And so, this is the mean square error calculation. And here's our model calculation, just w.X. So, here we're going to set a number of iterations and call it, loop over the iterations, and eventually we can plot how they did the error here. So, this is the cost or error function, loss function. So, it decreases as we iterate. And here, it's nearly zero. We've basically solved it. So, at the beginning we can take big steps, and as we get close to the solution, we need to take smaller steps and we'll talk about that later. So, let's just see this running. So, let me execute this, then this is the setup. So, I'm just testing against three points here, and this is the solution.

© MIT xPRO® 2024. All rights reserved.

Now, I'm going to do some plotting, so we'll come back and look at how the plotting is done, but it's fairly straightforward. And so here, now let me show you these plots, they're running. And what we see is we're doing a different number of iterations and we're choosing here different learning rates. So here this is 0.001 and it finds a quite a good solution in 200 steps. In 50 iterations, it's a little bit out. You can see here, it should be flat and it's not quite. And then if we just do 20 iterations, it doesn't it doesn't converge yet. So, we need to play around with the learning rate and the number of iterations, those are the things that we can control here. I'm just showing you how it converges for, we're taking 200 iterations for that one. So, the best model minimizes the least square error. And, this will give us our solution. Okay, so why don't you exercise that code, make sure you can understand how it converges, and then you'll see I've added some other code here which allows you to test out how it converges. We'll see the little plot out here. In this one, it doesn't get to the final solution. However, when the learning rate is 0.5, it does. It gets very close here, so you can see how it goes down those contours towards the solution, and the solution is in the middle here. Okay, so gradient descent, it's used in machine learning, in neural networks, and in many other places. Okay, bye for now.

Video 4: Visualizing Gradient Descent Using Matplotlib (10:09)

So, we saw in gradient descent that we started with w0 and w1 at some level and we chose to be w0, 0 and w1, 0. So, that meant the slope is 0 and the Y coordinate, the intercept is 0. So, this line at the bottom here is where we started, and then we stepped and we increased w0 and we increased w1. So, w0 went to about 0.25 and the slope increased. And the target was that y = 1, was our target. So, it iterated and eventually we got the line y = 1 as our solution. Let's see, we can visualize now what's happening. So, the best model minimizes the mean square error. So, we have points and here is a Y coordinate, that's some X. X is the independent variable and our Y predicted is here.

So, we have some error and we're going to actually square the errors and take the mean. So, we're going to add them all up and then divide by the number of points and that would be our mean square error. Now, let's see if we can visualize in the error space, the error that we're making, we sometimes call that the loss function. So, in the loss function here, we have that the loss is equal to Y is the actual Y. And then X.w transpose is the prediction that we're going to make by altering the w's. So, we can see that as we alter the w's, the loss is going to get bigger or smaller. And we note that the loss is quadratic, there's a square term there. So, we're going

© MIT xPRO® 2024. All rights reserved.

to move around the w0 and w1 space. So, what we're going to do is visualize the loss space and here we're plotting w0 along the horizontal and w1 up.

Now, we know our solution. In this case, I'm taking the y solution to be 2 but no slope, so w1 will be 0. So, here is our solution that w0 is 2 and w1 is 0. Now we start at w0, 0 and w1, 0. So, this is where we start and then we step. So, we have a learning rate and here what's happening is we're guessing that w0 needs to increase, but w1 increases as well. Then it reaches w1, realizes that no, it now needs to go and follow this path until it reaches w0 is 2 and w1 is 0. So, let's write the code so we can actually code this up. So, the first thing we need to do is to realize that we need to plot the loss function as we vary w0 and w1. So, w0 equals 2 is where we're trying to get to. And we start at this point here, w0 equals 0 and w1 equals 0. Let's see if we can write this code. We're going to load up into tools because this will help us plot a contour. Now, first we're going to create a grid. So, we use line space to add a resolution. We're going to choose a resolution and we're going to divide up the space, if you like into a grid. And then, so, here's our mesh grid that we can use and we get the xy coordinates and we set the Z.

So, that's our loss to 0 here. Now we're setting our w. So, this is our goal, this is where it's going to be 0 is at (2,0) in w space. Now we call our product here, and we're going to plot as Z. We're going to plot this function that we're going to pass in, which is going to be our cost function here. So, our cost function is a lambda function in Python where we pass in w, and then it'll tell us the different X's and Y's, what the cost function is. And so, we're taking the square of the predicted 1 - y, the actual one, and we're squaring it. So, this is our cost, and we're going to plot it the cost now at every point. So, this is how we get these contours here. We've got a grid. You can see the grid in the background and we're plotting those lines. So, as we move away from the solution where w0 is 2 and w1 is 0. As we move away, we're going to get these contours. So, these w's would have all the same error on that contour. So, that represents the error if you like, and it looks like a bowl. And, here we start at 0, 0 and we go down normal to these contours. And then, we hit the plane here and it takes, there are more contours that we can't see that we're not plotting here, but that it then iterates until it gets to this point. So, here's our cost function. Now, we're going to set up the problem, and this is the same as we did before. We're going to calculate the X's with, we're going to have a row of ones, and then we're going to have all the X's.

And then for the model, we're going to calculate the Y. And here, we're doing it all with w1

© MIT xPRO® 2024. All rights reserved.

equals 0, w2 equals 0. And now we say, I know, but our goal is y = y + 2. And this is where we call now the gradient descent. So, we're setting up the plot. We're going to loop over different learning rates. Here, I'm setting the learning rates as 0.01, 0.03, and 0.05. So, we're going to plot out three different learning rates. So, take a look at this code. And, here's the actual plotting. So, this is calling plot contour, that's just what we defined up here. So, we wrote it ourselves. So, that'll give us those contour lines. And now, here we're plotting the history. So, we're out of the w history, we're saying now this is the path we took in w space, and we're going to plot w0 first, and then w1. So, that gives us the red dots. You see, we're doing the red dots here, and then we do some labeling. So, let's run it. So, here we run and run again and we'll see down here our output. So, you'll see here when the learning rate is 0.01, we run out of iterations.

We get to the plane if you like, but we don't iterate to the correct solution. Now you see this one nearly hits the correct solution, and then with the learning rate of 0.05, we hit pretty much the correct solution. So, you can see we're going down perpendicular to these contour lines. And in fact, even here we're perpendicular to the contour lines. You can imagine other contour lines here that I'm not showing. Okay, so that's visualizing what's happening when we do gradient descent, that we're navigating some contour space. In this case, it's parabolic. We've got a square term in our loss function. In general, it'll be a more complicated space, but this allows you to understand what we're doing and how the learning rate affects the solution. Why don't you repeat this and choose different learning rates? So here, change these. Why don't you half the first one and then keep this one the same and then double the last one? So, we'll have 0.05, 0.03 and double that would be 0.1. So, why don't you run this with those different learning rates and submit the plots? Okay, bye for now.

Video 5: Introduction to Bayes Theorem (03:22)

So, are rich people happy? What do you think? A Harvard MBA student told us that only 10% of happy people are rich. Well, let's see if we can figure out what that means. We're going to talk about Bayes theorem. So, here's the background for it and the way that I remember it. So, in set theory, we've got two sets A and B, and their intersection is written as A intersection B. So, here are those two sets, and this is A intersection B. It's the area that's common to A, both A and B are in this.

Okay, so that's the mental model, and we can approach it two ways. We can say let's suppose A

© MIT xPRO® 2024. All rights reserved.

and B are events. So, these are both A and B have occurred if we're in this middle region. So, this is where event A has occurred and this is where event B has occurred. And then this would be if A has occurred and then B occurs, or if B has occurred and then A occurs. Let's take a look to see how we can write this. So, in probability notation, we say the probability of event A is P(A). So, that would be the area of the circle A. The probability of event B is P(B). Now the probability of B, given that A has already occurred, is written as P, B and then the vertical line A. And we read this as we're saying, this is the conditional probability that B occurs given that A has occurred, or the probability of B given A. So, the probability of A and B is probability of B given A. But that must be the same as the probability of A occurring given that B has occurred times the probability of B because they're both A intersection B.

So, this is Bayes theorem. Okay? That it's A intersection B and we can write it two ways, the probability of A given B and then the probability of B. And the probability of B given A times the probability of A. And just rearranging this, we get that the probability of B given A is the probability of A given B times the probability of B divided by the probability of A. Now this doesn't seem too useful. It just looks like trickery, but in fact it's very useful. So, let's go and see if we can solve the problem of are rich people happy. Bye for now.

Video 6: Bayes Theorem Exercise (02:19)

So, let's see if we can calculate what percentage of rich people are happy. The Harvard students said 10% of happy people are rich. So, let's draw our diagrams here. Here, we've got everyone, all people. The blue area are happy people and then the red area are rich people and then we have an intersection of rich and happy. Now that intersection can look like this on the left or it could look like this on the right and we don't have enough information to decide. But using Bayes theorem we can get an answer if we know a couple of extra things.

So, the MIT student thought for a moment and then googled the following. Of all the population, what percentage are happy? And google said 40% are happy. So, now we know that of all the people, H is 40%. And now he googled, what percentage of all people are rich? And, the answer was 5%. So, of all the people, the red area is 5%. So, what can we conclude from this? What do we want to know? Well, what we want to know is, given that we're rich, what's the probability that we're also happy? So, the prior is we are rich, and now the question is, are we happy? So, we want in some sense P H given R, if R is rich, what's the probability we're happy, H? So, go away

© MIT xPRO® 2024. All rights reserved.

and see if you can calculate now what percentage of rich people are happy. So, I will give you the answer in the next video. Bye for now.

Video 7: Bayes' Theorem Solution (02:51)

So, the Harvard MBA students said that only 10% of happy people are rich. And, the MIT student said I need more information, and googled and found out that 40% of all people are happy and 5% of all people are rich. So, what did the student actually do then? Well, the student is going to take advantage of Bayes and Bayes says that if we have a prior probability, so probability of something B and now we see that say A has occurred. And we ask now, what is the probability of B, given that A has already occurred? Our answer is probability of B given A, and the new information now gives us a new probability of B.

So, given that A has occurred, probability of A, and the probability of B given A, gives us a new probability for A given B. And this is how we write it. So, let's take a look at the Harvard student and the MIT student. What we really are asking is given that we are rich, what's the probability that we are happy? So, let's call rich R and happy H. We want to calculate P H given R.

So, here we use Bayes, and that's the same as P R given H, probability that we're rich, given that we're happy, probability that we're happy divided by the probability that we're rich. Only 10% of happy people are rich. So, given that we're happy, the probability that we're rich is 10%, 0.1. The probability that we're happy is 0.4. The probability that we're rich, just of all the people, just the normal of anyone in the population, is 0.05. Now when you divide this out, the answer is 0.8. So, 80% of rich people are happy, according to this data.

Now, I'm sure we'll have volunteers that would test this out for us. We'll make them rich and we'll see if they're happy. Okay, so that's the answer to how many rich people are happy. What's the percentage? 80% seems like they're a happy bunch. Okay, so hope you enjoyed the exercise. Bye for now.

Video 8: Applying Naïve Bayes Theorem to Detect Spam Email (01:43)

So, let's see how we apply Bayes rule to a real problem. Consider that we're receiving emails and we want to filter out the spam. So, we want to apply a label to the incoming email. So, we're going to decide based on some features of that email. So, those features are going to be the words contained in the email. So, we're trying to make a decision about the label that we're

© MIT xPRO® 2024. All rights reserved.

going to apply, and the two labels that we're going to go with are spam or normal, based on the features. Now, normally with Bayes, we'll have that we can flip this around now and say, given a certain label, how often does that feature occur?

So, what's the probability of that feature given the label times the probability of the label. Now, the probability of the feature, if we have two labels, we can divide one by the other and that probability of the feature will drop out. We'll be left with this equation here that I'm showing, where the probability of it being normal given these features divided by the probability that it's spam given these features is given by this right hand side. So, that's one insight in that we don't need to worry about this division here.

Video 9: Further Applications of Naïve Bayes Theorem (01:17)

Now, what do we do when we don't have words in common? So, we have password that occurs in spam but doesn't occur in normal mail. So, what we do is we add one occurrence to that dataset. So, we had 30 emails and so we have one word in 30 emails, 0.03. And then, similarly for lunch, we add it to the spam data and that gives us a probability of 1/20th. So now, if we want to calculate the probability for an email with password in it, we can work through, and we'll see that the probability is 1/10th.

So, you've seen how to handle calculating the probability for an email with the word password. Why don't you work through the probability for an email with the word lunch in it? Tell us what you think it should be. Should it be spam or is it a normal email? Bye for now.

Video 10: Using Naïve Bayes Theorem to Calculate Probabilities for Multiple Features (04:27)

Now, so far we've just assumed that we're only looking at one word in the email. So, that's not very realistic. The email will have many words in it. And so, the question is now, what do we do when we have lots of words? So, let's consider the case where we have two words. We have money and lunch. And so, we want to understand what's the probability of this email being normal or spam. So, one thing that we could do is we could say, well, given that this is a normal e-mail, what's the probability that money, that feature money is in the email and what's the probability that lunch is in that email? Now, if we assume that these probabilities are totally independent, and we'll see that that's typically what we will do, we'll assume that each feature

© MIT xPRO® 2024. All rights reserved.

occurs independently of the other.

So, we assume that, that feature is being drawn from an IID distribution, an independent identically distributed distribution. So, if they are independent, then we can just multiply the probabilities. Given that this is a normal email, the probability that feature one occurs, we can multiply by the probability that feature two occurs. And so for money and lunch, we would just do this calculation of these probabilities being multiplied. And what we'd see is that the probability, the relative probability of it being normal is 1/10. So, the conclusion would be that this is spam.

Now this assumption, that things are independent of each other is given the term Naive Bayes. And, what it means is that if we have a number of features, we can just multiply their probabilities by each other. So, here is the new equation where if we've got N features, then we have a product of N conditional probability distributions. So, probability of f1 given N times the probability of f2 given N, etc. So, you can imagine if we have 1,000 words, we've got a lot of multiplications here and these probabilities are all less than one and we can run into some difficulties computationally with underflow.

Now, we can get around that by instead of dealing with the probabilities, we can take the log of the probability and that's often done. And, so taking just either the numerator or the denominator here, we can express it in this way that taking the logs, then this becomes a sum. We use the property that log of A times B is equal to the log of A plus the log of B. So, now we have a summation which computationally is much more friendly.

Now, there's lots of other issues that will arise in this kind of computation, so if you want to understand more, I suggest you start reading at this link here, which takes you through what's called the maximum a posteriori for machine learning. Basically, it's dealing with these probability distributions and how we sample them. For data engineering, we may need to know some of these tricks, but not all of them. So, now why don't you calculate the posterior relative probability for an email with the words password and the money in it? Okay, bye for now.

Video 11: Introduction to Gaussian Naïve Bayes Theorem (05:22)

In this exercise, we're going to focus on using Gaussian Naive Bayes. The difference we have now is that we have distributions of our features. So, here we're going to use the height, weight,

© MIT xPRO® 2024. All rights reserved.

and foot size of a person to determine whether they're male or female. So, here's our training data. We've got four measurements for male and four for female. We're going to assume Gaussian distributions, so this is why we call it Gaussian Naive Bayes. And so, we're going to have for height, we're going to have one distribution for the males and a different distribution for the females.

So, you can see just by inspection of the data that the mean for the males is going to be different from the mean for the females for both height, weight, and foot size. Now, what we're going to rely upon for calculating the posterior probability is we're going to use the likelihood of seeing, for example, a person's weight. Let's suppose we have the person and they have a weight of 114 lbs. Then we'd use the Gaussian distribution that we have. This one is close for a female, but the male would be different to get us a probability. And here, we're getting a value of 0.04.

So, we take the value and this gives us the likelihood of seeing that value based on this probability density function on this Gaussian distribution. So, we can easily calculate the Gaussian distributions because all we need is the mean and the variance. So, step one is to write code to calculate the mean and the variance of each of the six Gaussian distributions. So, the six Gaussian distributions are for male. We have a distribution for height, so just those four values. We need the mean and the variance. Then for weight, again, mean and variance and for foot size. And then we have the same for the female.

So, we have a total of six Gaussian distributions that you're going to calculate. Now, here's our test data. We've got an unknown person. We don't know if they're male or female, but they've got a height of six foot, a weight of 130, and a foot size of eight. So, what you need to do is now to calculate each of the likelihoods. So here, this is the likelihood of seeing a certain height for either a male or female. So, X we have to substitute either the male Gaussian or the female Gaussian. Then what we can do is, as we saw before, because the heights and weights and foot sizes are assumed to be totally independent, then we can just multiply these probabilities.

So, this is, given that this is a female, this is the probability of seeing this height. So, it would be six, the height would be six here. And then the probability of seeing the weight of this person if it's female and we can compare that with the same probabilities for whether it's a male. And then comparing these two posterior, we can decide whether it's more likely that this person is a

© MIT xPRO® 2024. All rights reserved.

male or female. So, I'm going to give you sample code here for calculating each one of these. So, given that it's a female distribution and we see this height, what's the probability density function that we should insert here? So, here's the code.

So, this takes a mean and a variance. So, you need to plug in the ones that you've calculated. So, you've got all six of those to plug in. And then this would be the feature that you're looking at. And this will give you the probability of seeing that feature, the likelihood, in this case, I'm saying what's the likelihood of the person being 5 foot 8 or 5.8 feet when we have this Gaussian distribution? So, you need to calculate six of these to plug into these formula here. So, then you can calculate the posterior probabilities for male or female and tell me the likely gender for this person. Good luck. Bye for now.

© MIT xPRO® 2024. All rights reserved.